

Cross-Layer Security Validation Header

A proposed header, metadata, cryptographic-binding, interoperability, and standardization design for adaptive cross-layer cyberattack detection and prevention

Purpose. This document develops the header proposed in the research concept into an implementable specification outline. It distinguishes the logical security object from packets and files, introduces extensible header fields and record types, explains lifecycle and enforcement, and gives a practical path toward standards adoption.

Scope

The Cross-Layer Security Validation Header (CLSVH) is not intended to replace IP, TCP, TLS, HTTP, file formats, endpoint telemetry, or existing security products. It is a compact, authenticated metadata envelope and a companion evidence record that let trusted components correlate a logical object across network, application, filesystem, and memory observations.

Design principles

1. The sender **MUST NOT** self-declare trusted security status. A trusted validation authority issues security assertions. 2. The design **SHOULD** store large evidence off-object and carry only compact references or hashes in the header. 3. Privacy-sensitive identifiers **SHOULD** be scoped, pseudonymous, or encrypted when crossing trust boundaries. 4. The base format **MUST** be versioned and extensible. 5. A validation result is time- and event-bounded; it is not permanent trust.

1. Object Model

A practical design needs more than one identifier. A packet, a downloaded file, a process, and mapped executable pages are different technical objects. CLSVH uses an object lineage model so that observations can be linked without falsely claiming they are identical byte-for-byte.

Identifier	Meaning	Example use
SOID — Security Object ID	Stable identifier for one logical security object or lineage node. Generated by a trusted issuer.	A downloaded document and its initial stored file share an SOID.
CID — Content ID	Cryptographic digest of a particular byte sequence. Changes whenever content changes.	SHA-256 digest of the file received after download.
OID — Observation ID	Uniquely identifies one layer-specific observation or validation event.	A Zeek network event, an eBPF file-open event, or a process execution event.
Parent SOID	Links an object to its direct precursor or source.	A macro-extracted payload is derived from the original document.
Instance ID	Identifies a concrete runtime or storage instance.	Filesystem inode + mount namespace; process PID + start time; socket tuple.

Why this matters

A content hash alone cannot represent lineage: a document may be unpacked, decoded, copied, edited, or used to create a child process. Conversely, one SOID does not prove identical content. The design should carry both an SOID for provenance/correlation and a content digest for exact integrity checking.

Recommended lifecycle

Ingest → Identify → Observe → Validate → Bind → Enforce → Revalidate → Close. At ingest, a trusted security gateway assigns an SOID. Each layer emits an evidence record referencing that SOID. A policy engine evaluates the current evidence and issues a signed or MAC-protected validation assertion. The enforcement point permits, monitors, quarantines, or blocks the next action. Material change creates either a revalidation event or a derived child SOID.

2. Header Architecture

Use a two-part design: (A) a compact in-band or sidecar header for fast decisions; and (B) an out-of-band evidence record in a trusted metadata store. This avoids embedding large telemetry, personally sensitive data, or mutable policy logic inside every packet or file.

Transport profiles

Profile	Where it travels	Best use	Caution
Sidecar / reference profile	Security metadata service, endpoint agent, XDP/eBPF map, database, message bus	Recommended starting point; works even when application protocols cannot change	Requires reliable lookup and access control
Application metadata profile	HTTP/SMTP/object-storage extension, manifest, message envelope	Controlled application ecosystems and gateways	Intermediaries may strip or alter unprotected fields
Encapsulation profile	A new tunnel/overlay extension used only between trusted nodes	Private networks, controlled service meshes, security appliances	Do not alter general Internet packet headers without compatibility study
Filesystem xattr / label profile	Extended attribute, security label, immutable metadata store	File admission and endpoint enforcement	Copy, archive, cloud sync, and format conversion may lose metadata
Runtime event profile	eBPF/LSM/audit/EDR event linked to the SOID	Process, memory, file, and network correlation on hosts	Requires kernel/agent trust and careful performance design

Recommendation

For the first prototype, implement the sidecar/reference profile plus Linux runtime events. Treat in-band transmission as an optional extension only for paths where all intermediaries are controlled. This reduces protocol ossification, avoids middlebox breakage, and makes the concept testable without changing Internet protocols.

Header formats

Define a canonical logical schema first; serialize it using a compact binary format for high-rate data paths and JSON only for debugging or administrative APIs. CBOR is a strong candidate for compact encoding, while COSE can provide standardized signing, MAC, encryption, and protected-header mechanisms. COSE explicitly supports header parameters carried with signatures or MACs, including authenticated or unprotected attributes. [web:47]

3. CLSVH Core Header

The following is a proposed version-1 logical header. It is deliberately compact; detailed telemetry stays in the evidence store.

Field	Type / size	Required?	Purpose and processing
Version	uint8	MUST	Schema version. Receivers MUST reject unsupported critical versions.
Flags	uint16	MUST	Bit flags: critical-extension present, derived object, quarantined, privacy-protected, replay-protected.
SOID	128-bit opaque value	MUST	Trusted logical-object identifier; not derived directly from untrusted sender input.
Parent SOID	128-bit or null	SHOULD	Direct lineage source. Null only for root objects.
Object class	uint8 enum	MUST	Packet flow, message, file, archive member, process, memory mapping, script, container artifact, other.
Layer / context	uint8 + enum	MUST	Network, application, filesystem, process, memory; plus protocol/context identifiers.
Content digest	algorithm ID + digest	SHOULD	Exact content binding where bytes exist; for streaming data, include segment/chunk semantics.
State	uint8 enum	MUST	UNKNOWN, PENDING, VALIDATED, SUSPICIOUS, QUARANTINED, MALICIOUS, REVOKED, EXPIRED.
Risk score	uint8 or uint16	SHOULD	Normalized score; MUST state scoring profile/version to avoid semantic ambiguity.
Validation lifetime	not-before, expires-at, event mask	MUST	Time limit plus events that invalidate the assertion.
Evidence reference	URI/opaque ID + hash	SHOULD	Reference to trusted evidence record and its immutable digest.
Issuer ID / key ID	opaque ID	MUST if authenticated	Identifies the trusted component and verification key.
Sequence / nonce	uint64/96-bit	MUST	Provides anti-replay and ordering within issuer scope.
Previous binding	digest	SHOULD	Hash of the prior validation assertion for append-only lineage binding.
Authentication	COSE Sign1, COSE MAC0, or equivalent	MUST	Protects all critical fields, issuer, sequence, and previous binding.

Critical versus optional fields

A receiver MUST understand all fields marked critical before relying on the assertion. Optional fields can improve visibility but cannot weaken a deny/quarantine decision. New fields should use a type-length-value (TLV) extension mechanism, with a criticality bit and an IANA-style registry for field types and state values.

4. Additional Header Characteristics

The following characteristics make CLSVH usable across multiple environments rather than only as a static “security header.”

Characteristic	How it works	Why it is needed
Extensibility (TLV / CBOR map)	Unknown non-critical extensions are ignored; unknown critical extensions cause a safe fallback such as PENDING or REVALIDATE.	Permits future fields without breaking older implementations.
Canonical encoding	The authenticated byte representation MUST be deterministic.	Two encodings of the same logical data must not produce signature-verification ambiguity.
Authenticated protection	Place SOID, state, expiry, issuer, sequence, evidence hash, and previous binding in protected/authenticated fields.	Prevents attackers from altering a “malicious” state into “validated.”
Selective disclosure	Keep sensitive evidence external; optionally encrypt selected attributes for authorized recipients.	Supports privacy, data minimization, and multi-organization sharing.
Issuer scoping	SOID uniqueness is within an issuer namespace; include Issuer ID.	Avoids collisions and makes trust decisions traceable.
Replay protection	Use a nonce or sequence plus expiry and a per-issuer replay cache.	Blocks reuse of a formerly valid assertion after conditions change.
Revocation	Allow a status reference or revocation event tied to SOID/Assertion ID.	A signed assertion may need withdrawal before natural expiry.
Delegation	Explicitly constrain which validation authority may issue which assertion types.	Prevents a low-trust sensor from issuing a final “allow” decision.
Policy profile ID	Reference a versioned policy/scoring profile.	A score of 70 must have a consistent meaning across products.
Auditability	Store immutable evidence digest, issuer, time, action, and reason code.	Supports incident response and reproducible research.
Fail-safe behavior	If authenticity, freshness, or required evidence cannot be verified, use policy-defined PENDING, REVALIDATE, or QUARANTINE.	Avoids treating verification failure as success.

Do not use a raw checksum as protection

A checksum can detect accidental corruption but does not establish who created the header. Use a keyed MAC for a shared trusted domain or a digital signature for multi-party verification. HMAC is a standardized keyed-hash message-authentication construction; its assurance depends on the key and chosen hash function. [web:50][web:57]

5. Cryptographic Binding

Each validation event creates a binding record. The record commits to the object identity, canonicalized critical metadata, current evidence digest, previous binding, issuer identity, and freshness values.

Suggested binding input

```
binding_i = H(canonical(SOID || ParentSOID || ObjectClass || Layer || State || RiskProfile || Risk ||  
ContentDigest || EvidenceHash || NotBefore || ExpiresAt || EventMask || IssuerID || Sequence ||  
binding_(i-1)))
```

The issuer then authenticates this binding. In a single enterprise trust domain, a COSE_Mac0/HMAC construction may be adequate. Where multiple independent organizations or devices must verify the record, use COSE_Sign1 with asymmetric keys and a documented certificate or trust-anchor model. CBOR/COSE is useful because it supports compact representation and standardized signatures, MACs, encryption, protected headers, and countersignatures. [web:47][web:49]

Threat protections

Threat	Required control
Field modification	Authenticated protected header and evidence digest
Replay of old "valid" assertion	Expiry, sequence/nonce, revocation, and event-triggered invalidation
Lineage deletion or substitution	Previous-binding hash plus append-only evidence store
Unauthorized issuer	Issuer allow-list, key management, authorization/delegation policy
Evidence-store alteration	Content-addressed evidence digest, access control, retention audit
Downgrade via unknown extension	Critical-extension flag and fail-safe receiver behavior
Compromised endpoint agent	Measured boot/attestation where feasible, least privilege, protected key storage, independent sensors

Security Validation Lifetime (SVL)

SVL should be a compound rule, not only a timestamp. A validation assertion is usable only when: current time is within its validity window; the sequence is fresh; no revocation exists; the content digest still matches when applicable; and no invalidating event has occurred. Typical invalidating events include content modification, a change of user or privilege, process-image change, archive extraction, protocol/zone transition, suspicious behavior, or policy-profile change.

6. End-to-End Example

Scenario: a document arrives through HTTPS, is written to disk, opened by an office application, and launches a suspicious child process.

Stage	Event and header action	Policy consequence
1. Network ingress	Gateway creates root SOID A, records TLS/session and download evidence, sets State=PENDING.	Allow controlled transfer or stage in temporary storage.
2. File assembly	Endpoint computes CID for completed file, records file type and malware scan result, links to SOID A.	Issue an admission assertion only if structure, policy, and integrity checks pass.
3. File admission	Filesystem record stores SOID A, CID, assertion reference, expiry, issuer, and protected authentication tag.	Permit normal storage; mark file for revalidation on modification.
4. Application open	Runtime agent observes application opening the file and emits OID linked to SOID A.	Risk engine combines file and process context.
5. Child process	Document causes a script interpreter or shell to launch. Create child SOID B with ParentSOID=A if it is a derived object; preserve evidence link.	Raise risk; invoke deeper inspection or sandboxing.
6. Memory/execution	Runtime sensor records executable mapping or suspicious injection behavior linked to B.	Set B=SUSPICIOUS or MALICIOUS; potentially revoke A's "validated" use for execution context.
7. Enforcement	Policy engine signs an updated assertion with a new binding and sequence.	Quarantine file, terminate/contain process, block network egress, retain selected evidence.

Key rule

Do not simply overwrite history. Append a new assertion with its own issuer, evidence digest, sequence, and previous-binding reference. This makes security state changes inspectable and resistant to silent substitution.

7. Standardization Strategy

A standard should begin as an interoperable data model and implementation profile, not as a request to add a universal Internet packet header. Internet protocols depend heavily on compatibility with endpoints, middleboxes, encrypted transport, and existing registries. RFC 6709 specifically advises protocol designers to make extension mechanisms explicit and consider their architectural implications. [web:23]

Phase 1 — Research specification

Publish a versioned CLSVH data model: terminology, threat model, canonical encoding, required/optional fields, state machine, lifecycle rules, security/privacy considerations, and conformance tests. Use normative language consistently: BCP 14 defines the special meanings of uppercase MUST, SHOULD, MAY, and related terms. [web:17][web:18]

Phase 2 — Reference implementation

Build at least two independent implementations: for example, a Linux endpoint agent using eBPF/LSM/audit signals and a network gateway/collector. Demonstrate sidecar metadata, file-admission enforcement, authenticated assertions, event-based revalidation, and evidence export. Measure latency, event loss, CPU/RAM, detection impact, false positives, and failure behavior.

Phase 3 — Interoperability profile

Define a common CBOR/COSE profile, deterministic test vectors, public sample traces, conformance test suite, and a registry specification for object classes, state values, extension IDs, reason codes, and algorithm identifiers. A W3C PROV mapping can improve research interoperability: PROV models provenance in terms of entities, activities, agents, and derivations. [web:32][web:38]

Phase 4 — Community and standards venue

Engage relevant technical communities based on scope: IETF for an Internet interchange or security-token format; OpenSSF or CNCF for software-supply-chain/cloud-native integration; OASIS for structured security-information exchange; Linux security/eBPF communities for endpoint implementation; and academic provenance-security communities for evaluation. Start with an Internet-Draft only after problem statement, use cases, interoperable encoding, and implementation evidence are mature.

Phase 5 — IETF path, if applicable

An individual Internet-Draft can begin discussion, then may be adopted by an appropriate working group if there is consensus and scope fit. The draft must state security considerations and any required IANA actions; author guidance specifically calls for meaningful security analysis and explicit IANA considerations. Historical IETF process guidance is documented in RFC 2026, while current process updates should be checked as standards work begins. [web:31][web:34][web:33]

8. Proposed Registries and Conformance

A standard needs shared value spaces so independent products interpret the same header consistently.

Registry	Example values	Suggested policy
CLSVH Version	1, 2, experimental	Standards action for major versions; documented experimental range
Object Class	network-flow, message, file, archive-member, process, memory-map, container-image	Specification required
Layer / Evidence Type	L2/L3/L4/L7, file-create, file-write, exec, mmap, privilege-change	Specification required
Security State	UNKNOWN, PENDING, VALIDATED, SUSPICIOUS, QUARANTINED, MALICIOUS, REVOKED, EXPIRED	Standards action; avoid overlapping semantics
Reason Code	digest-mismatch, expired, behavior-anomaly, policy-deny, replay-detected	Expert review
Invalidation Event	modified, privilege-change, zone-change, process-change, revocation	Expert review
Critical Extensions	extension type + semantics	Standards action
Cryptographic Algorithms	COSE algorithm identifiers where possible	Reuse existing COSE/IANA registries
Risk Scoring Profile	profile identifier, version, thresholds	Specification required; avoid universal score meaning without profile

Minimum conformance requirements

A conforming issuer **MUST** generate unique SOIDs in its issuer namespace, protect all critical metadata cryptographically, include freshness/replay controls, publish its supported profile and algorithms, and retain sufficient evidence to justify enforcement. A conforming verifier **MUST** validate issuer trust, authentication, expiry, sequence/nonce, required critical extensions, and policy applicability before accepting an assertion. A conforming enforcement point **MUST** log the reason code and applied action.

State machine

UNKNOWN → **PENDING** → **VALIDATED** is allowed only after required checks succeed. Any material change, failed verification, or suspicious signal transitions the object to **REVALIDATE/PENDING**, **SUSPICIOUS**, or **QUARANTINED** according to policy. **MALICIOUS** and **REVOKED** are terminal for the relevant policy scope unless a separately authorized remediation workflow creates a new assertion.

9. Research Topics and Links

Use these topics as separate literature-review sections. The links below are starting points, not proof that the complete CLSVH combination is novel.

Topic	Why it relates to CLSVH	Research link
Zero-trust architecture and continuous authorization	NIST SP 800-207, Zero Trust Architecture — resource-centric protection and dynamic authorization.	Open source [web:4]
Zero-trust implementation and maturity	CISA Zero Trust Maturity Model v2.0 — maturity pillars and cross-cutting capabilities.	Open source [web:2]
System provenance and object lineage	W3C PROV-DM — entities, activities, agents, and derivations for provenance representation.	Open source [web:32]
Endpoint provenance with identifiers	IBM Research / LSS NA: Provenance-Aware Integrity Monitoring with Linux Security Identifiers — correlating process, file, and network flows.	Open source [web:15]
Authenticated compact metadata	RFC 9052: CBOR Object Signing and Encryption (COSE) — compact signing, MAC, encryption, and header parameters.	Open source [web:47]
Countersignatures and multi-party attestations	RFC 9338: COSE countersignatures — a second party can sign protected data.	Open source [web:49]
Message authentication	RFC 2104: HMAC — keyed hashing for message authentication.	Open source [web:50]
Secure manifests and protected metadata	RFC 9124: Manifest Information Model for Firmware Updates — concise, machine-processable, protected metadata.	Open source [web:48]
Protocol extension design	RFC 6709: Design Considerations for Protocol Extensions.	Open source [web:23]
Normative standards language	BCP 14: RFC 2119 and RFC 8174 — meanings of uppercase MUST, SHOULD, MAY.	Open source [web:22]
Internet standards process	RFC 2026: The Internet Standards Process; check the IETF process site and current updates before submission.	Open source [web:34]
Internet-Draft author guidance	IETF Guidelines to Authors of Internet-Drafts — security and IANA considerations.	Open source [web:31]

10. Candidate Research Questions

- RQ1: Does authenticated cross-layer object lineage improve detection of attacks that appear benign in isolated network, file, process, or memory telemetry?
- RQ2: Which identifier model — SOID only, content digest only, or SOID + content digest + parent lineage — produces the best correlation accuracy under object transformations?
- RQ3: What is the latency, CPU, memory, and storage cost of creating and verifying CLSVH assertions at different traffic and endpoint-event rates?
- RQ4: Does event-based Security Validation Lifetime reduce stale “valid” decisions compared with time-only expiry?
- RQ5: Can a sidecar metadata profile interoperate across a network gateway and Linux endpoint without altering application protocols?
- RQ6: Which authenticated format — HMAC/MAC within one domain versus signatures/countersignatures across domains — gives acceptable performance and trust separation?
- RQ7: What false-positive and false-negative effects arise from lineage branching, copying, archive extraction, shared libraries, and process inheritance?
- RQ8: What privacy-preserving minimum metadata is necessary for effective enforcement and forensic reconstruction?

Suggested evaluation metrics

Correlation precision/recall; detection rate; false-positive and false-negative rate; time-to-decision; revalidation latency; header size; verification throughput; evidence-store growth; CPU and RAM overhead; event loss; quarantine accuracy; and resilience to tampering, replay, and rollback attempts.

Practical prototype scope

Start with one ingress path (for example, HTTPS file download), one endpoint operating system (Linux), one file-admission mechanism, one runtime sensor path (eBPF/audit/LSM), an external metadata store, and a rule-based risk engine. Do not attempt universal packet-level adoption, arbitrary application support, or full memory forensics in the first prototype.

Conclusion

The CLSVH concept becomes stronger when framed as an extensible, authenticated security-object lineage and validation-assertion format. Its practical value is not a new header placed indiscriminately into every packet. It is a trusted correlation contract across sensors and enforcement points: identify the object, bind evidence and integrity, limit validation lifetime, preserve derivation history, and make adaptive decisions that can be independently verified.